

Lab 4 - Odometry, Mapping, and Navigation with the TurtleBot3*

EECS/ME/BIOE C106A/206A Fall 2026

Goals

By the end of this lab you should be able to:

- Bring up a TurtleBot3 and interpret its LiDAR and IMU data
- Explain why wheel odometry drifts, and measure that drift on a real robot
- Write a ROS 2 launch file that starts and configures several nodes at once
- Use SLAM to build a map, and explain the `map` $\rightarrow$ `odom` $\rightarrow$ `base_footprint` frame chain
- Send navigation goals programmatically using a ROS 2 action client
- Explain how Nav2 plans, replans, and recovers when navigation fails

Relevant Tutorials and Documentation:

- ROS 2 tf2 package: <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Tf2.html>
- ROS 2 launch: <https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Launch-Main.html>
- ROS 2 actions: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>
- Nav2: <https://docs.nav2.org/>
- ArUco tag detection: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html

Contents

0	Initial Setup	2
1	Introduction to the TurtleBots	5
1.1	How do I turn this thing on?	5
1.2	Reading the IMU	5
1.3	The Camera	5
1.4	The Odometer	6
1.5	Driving the TurtleBot	6
1.6	Dead Reckoning and Breaking the Odometer	6
2	Open Loop Control	7
2.1	What you are writing	7
2.2	Open Loop Control Error	7
2.3	Running it	7

*Rewritten Fall 2026. Developed by Janus Sucharitakul, Christopher-Michael Garcia, Fall 2026.

3	Launch Files	8
3.1	SLAM	8
3.2	Anatomy of a Launch File	8
3.3	Writing <code>slam.launch.py</code>	8
3.4	Saving the Map	9
3.5	Going Further	9
4	Navigation with Nav2	10
4.1	Bringing up Nav2	10
4.2	Actions	10
4.3	TF and Coordinate Frames	11
4.4	Finding a Goal with ArUco Tags	12
4.5	TF Frame Broadcasting	12
4.6	Running the Tag Detector	13
4.7	Writing <code>nav_client.py</code>	13
4.8	Watching it Drive	14
4.9	Something in the Way	14
5	Before you Leave	15

0 Initial Setup

Before beginning this lab, we need to standardize everyone’s development environment. **This setup is required. Please follow every step carefully before continuing.**

Lab Computer and Partner Setup

Beginning with this lab, you may no longer use VM/RDP on your personal computer. All remaining labs must be completed using the lab computers.

You should also choose a partner that you will work with for the remainder of the course. Pick one partner’s account to use on the lab computer, log into that account, and complete the following setup together.

Also, if you haven’t pushed your current lab 2 code yet, please do so before continuing with these instructions.

Create a Fresh ROS2 Distrobox

Open a terminal on the lab computer. Check which Distroboxes currently exist:

```
distrobox list
```

Remove each existing Distrobox using:

```
distrobox rm -f <container_name>
```

Then create the full ROS2 container using:

```
inst-containers-setup
~ee106a/create-ros2-container
```

Enter the new container with:

```
distrobox enter ros2
```

Set up SSH

Configure your Git identity:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

Next, generate an SSH key pair:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

When prompted for a location to save the key, press **Enter** to use the default location. Just press **Enter** twice more when prompted to enter a passphrase. Then start the SSH agent and add your newly created key:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Now print your **public** SSH key:

```
cat ~/.ssh/id_ed25519.pub
```

Copy the **entire** output. Then, open your browser and go to:

GitHub → Settings → SSH and GPG keys → New SSH key

Name the key **EECS 106A Lab Machine**, set it as an **Authentication Key**, paste your public key into the key field, and save it.

Finally, verify that SSH authentication is working:

```
ssh -T git@github.com
```

The first time you connect, you may be asked whether you trust GitHub's host key. Type **yes**. If everything is configured correctly, you should see a message similar to:

```
Hi USERNAME! You've successfully authenticated,
but GitHub does not provide shell access.
```

Clone Your Personal Repository

Inside the **ros2** Distrobox, clone the private GitHub repository you created during the previous labs:

```
cd ~
git clone git@github.com:GITHUB_USERNAME/YOUR_REPO.git ros_workspaces
cd ~/ros_workspaces
```

Replace **GITHUB_USERNAME** with your GitHub username and **YOUR_REPO** with the name of your personal GitHub repository. **Next, go on GitHub and share your personal repository with your partner.**

Add the Fall 2026 Starter Repository

Next, add the Fall 2026 course repository as a second remote:

```
git remote add starter https://github.com/ucb-ee106/106a-student-labs-starter.git
git fetch starter
```

Your repository now has two remotes:

- **origin**: your personal GitHub repository
- **starter**: the official course starter repository

Throughout the rest of the course, you will use **starter** to retrieve the newest version of each lab while continuing to push your completed work to **origin**.

Clean Up Your Repository

Due to mistakes on our end, your personal repository accidentally contains outdated versions of the labs. Before continuing, remove those directories from your repository:

```
git rm -rf --ignore-unmatch -- lab3 lab3_ur7e lab4 lab5 lab6 lab7 lab8
git restore --source=starter/main --staged --worktree -- lab3 lab4
git commit -m "Update Labs 3 and 4 for Fall 2026"
git push origin main
```

Set Up an IDE

For the remainder of the course, please use **Visual Studio Code** (or another IDE) for development rather than editing directly with **vim**, **nano**, or **EMACS**. Open your lab workspace in VS Code and install the following extensions:

- Python — Microsoft
- C/C++ Extension Pack — Microsoft
- CMake Tools — Microsoft
- XML — Red Hat
- YAML — Red Hat

Along with that, as you work, use **Ctrl+Shift+T** to open new terminal tabs instead of creating many separate terminal windows. You may also use the integrated terminal in VS Code.

Checkpoint 0

Submit a checkoff request at <https://tinyurl.com/ee106a-fa26lab>. Before proceeding with the lab, you must show a TA that:

- You set up your Distrobox with the **ros_workspaces** folder with both the origin and starter remotes, and SSH setup
 - VS Code is set up for development
 - You have completed the **Robot Usage Quiz** on Gradescope
-

1 Introduction to the TurtleBots

TurtleBot is one of the classic platforms for mobile robotics research and teaching. The TurtleBot 3 is a *tank drive* robot, where the wheels on each side are controlled independently. By spinning the two wheels at different speeds or directions, the turtlebot is able to turn as well as translate. The TurtleBots we use in class have a 360-degree LiDAR for depth information, an RGB camera, a 6-axis IMU, and an onboard Raspberry Pi running ROS 2. The two drive wheels have encoders that measure how far each wheel has turned.

We have six TurtleBots available, each with a unique name: **kiwi**, **lemon**, **apple**, **mango**, **cherry**, **banana**.

1.1 How do I turn this thing on?

Open up a terminal tab. You'll ssh into the TurtleBot and run the following commands here.

Before you can listen for messages or give commands to the TurtleBot, you'll have to turn it on. Unplug your TurtleBot3's battery pack from the charging station, plug it into the TurtleBot, and turn on the switch on the side opposite the blue battery pack. Before you ssh into the TurtleBot, we suggest pinging your TurtleBot to ensure it is turned on. This can be done with the following command, where "fruitname" refers to the specific fruit name on the wood plate on top of your TurtleBot.

```
ping fruitname
```

When you start to see **64 bytes from ...** that means the workstation can reach the TurtleBot. Note that it may take a minute or two to do so. Now you can ssh into the TurtleBot using the login info below:

```
ssh fruitname@fruitname
Password: fruitname2025
```

If it's your first time ssh'ing you might see **Are you sure you want to continue connecting (yes/no/[fingerprint])?** just type **yes**

Now run the TurtleBot *bringup* sequence in **ssh**:

```
ros2 launch turtlebot3_bringup robot.launch.py
```

The TurtleBot is now launched, along with all of its sensors, and it is ready to receive motion commands. When you're done using a TurtleBot, you can close the ssh connection by typing **exit** in the command line. Please be sure to do this at the end of the lab!

1.2 Reading the IMU

An IMU combines accelerometers and gyroscopes to measure the robot's acceleration and rotational velocity. Some IMUs also include a magnetometer (ie, a compass). A 6-axis IMU contains 3 accelerometers and 3 gyroscopes. Accelerometers measure acceleration along one axis (so three measure along *x*, *y*, and *z*) while gyros measure rotational velocity about one axis.

See if you can figure out which topic the TurtleBot's IMU is published on. Once you have found it, echo it on the terminal and try to work out what the numbers correspond to. Read this while driving the bot (see below)

1.3 The Camera

The TurtleBot has an RGB camera as well as the LiDAR. You will use it in the last section of this lab to find an ArUco tag; for now, just confirm it works and see what it sees.

The camera is plugged into the Raspberry Pi, so it has to be started **on the TurtleBot**. In a second ssh session, leaving your bringup running in the first:

```
ros2 launch usb_cam usb_cam.launch.py
```

Now look at the images **on the workstation**:

```
rviz2
```

Click **Add** at the bottom left, choose **By topic**, and pick **Image** under `/image_raw`. You should see a live view from the robot.

At the top left where it says map, click the drop down and change it to odom to see the robot and remove the error.

1.4 The Odometer

The TurtleBot's two drive wheels each have an **encoder**, which is a sensor that records how much their specific wheel rotated.

From those two counts you can work out how the robot moved. If both wheels turn the same amount, it drove straight, and the distance is the wheel rotation times the wheel radius. If one turns more than the other, it curved, and how sharply depends on the difference between the two counts and the distance between the wheels. Do that continuously and add up the result, and you have an estimate of how far the robot has travelled and which way it is now facing. This is **odometry**.

ROS publishes that estimate on `/odom`. To view it:

```
ros2 topic echo /odom
```

The message is long, but the part that matters is `pose.pose.position` and `pose.pose.orientation`: where the robot thinks it is, and which way it thinks it is facing. You can drive the robot around with teleop and watch those numbers change.

1.5 Driving the TurtleBot

TurtleBot commands are sent over the topic `/cmd_vel`. Later you will command the robot from your own code, but for now use the built-in keyboard teleoperation node. Open a new terminal **on the workstation** and run:

```
ros2 run turtlebot3_teleop teleop_keyboard
```

Drive the TurtleBot around. Confirm that it tracks roughly straight when you press forward and backward. If it curves noticeably, flag a TA before continuing. What happens to the IMU messages while you are driving, especially when you start and stop?

Important! Make sure to use `teleop_keyboard` to move the TurtleBot. Do not pick up the TurtleBot while it's on. Doing so will mess up the `/odom`. To reset it, rerun the bringup sequence.

1.6 Dead Reckoning and Breaking the Odometer

Odometry works by starting from a known position (zero), then adding up new measurements at every timestep to estimate the current position. This sort of measurement is called **dead reckoning**.

Now, unfortunately, the odometer is not perfect. Since the encoders are reading the rotation of the wheels, minute errors can accumulate over time. Each estimate of where the robot is gets added to the previous estimate, without any way to correct itself. The longer the robot moves, the more error accumulates. This sort of error accumulation is always a consequence of dead reckoning.

Oftentimes, this is caused by wheel slip. Remember, the encoders measure the wheel rotation, not the robot's motion as a whole and the two are only equal for as long as the wheel keeps a grip on the floor. A wheel that slips will turn without moving the robot, and now the encoder will report an erroneous displacement.

With this in mind, let's go "break" our odometer!

Using teleop, **spin the robot in place for 15–20 full rotations**, and then bring it back to face its original direction. Press `s` to stop as needed. The odometry will have slipped a little throughout the process.

It should be noted that the encoders on these bots are very good. Furthermore, the turtlebot odometry uses the the IMU to reduce error during turns (this is still dead reckoning, since it's integrating, ie adding, linear acceleration and angular velocity measurements). Over the distances you will drive in this room, the accumulated error is small – often only a few centimeters at most. Do not expect the numbers to be wildly wrong. What matters is that the error exists, that it only grows, and that the robot cannot tell. On a longer route, or a slicker floor, these errors will accumulate into harsher consequences.

2 Open Loop Control

Now that you know a little about how the TurtleBot works, it is time to start controlling it through your own code.

2.1 What you are writing

Write a node `square_drive.py` in the `turtlebot_controller` package that drives the TurtleBot in a **1 m square** using only timed velocity commands, then stops.

```
ros2 run turtlebot_controller square_drive
```

You already know how to publish to a topic from Labs 1 and 2. You can choose the values of the variables but the TurtleBot caps out at 0.22m/s for hardware longevity.

To show odometry error, we provide code that subscribes to `/odom`. However, **your node must not use the odometry to steer**. This code is to show what the robot thinks the odometry output is.

2.2 Open Loop Control Error

You've implemented something called *open-loop control*, where you've sent the robot commands without actually checking those commands have been followed accurately. For example, you may have told the robot to go 0.1m/s for 10s, but it may have actually gone 0.95m/s for those 10s, leaving you 5cm short. This sort of error is called *open-loop control error*. Because you're not incorporating position measurements into your controller, you won't account for the error.

This may seem similar to the dead reckoning error you saw before, but it comes from somewhere else.

2.3 Running it

1. Drive the robot to start from the X
2. Stop the `teleop_keyboard` node. Then run your node and let it complete all four sides to return to the X
3. Read your node's printed output and measure how far you've drifted from the X. How do you know how much error came from open-loop control versus odometry?

Checkpoint 1

Submit a checkoff request at <https://tinyurl.com/ee106a-fa261ab>. At this point you should be able to:

- Demonstrate `square_drive.py`
 - Explain the function of the function `spin_for()`. Why does its duration matter?
 - Explain why spinning the robot ten times made things worse
 - Explain the difference between dead reckoning error and open-loop control error.
-

3 Launch Files

Until now, you have been opening a new terminal for every node. As projects get bigger, that gets inconvenient pretty quickly. In the next section, we will be running more than a dozen nodes in a particular order. That's where launch files come in.

A **launch file** is a Python script that describes what nodes will be run, with what arguments, and when. Unlike a Python script, it will only be executed with `ros2 launch`. In this section, you will build a launch file to launch the nodes necessary to run the SLAM process.

3.1 SLAM

SLAM stands for **Simultaneous Localization and Mapping**. It is a method where the robot builds a map of an environment (mapping) while at the same time working out where it is within that map (localization). Cartographer, which you are about to launch, is one implementation of it.

Think about walking through a neighborhood you have never been in. The first time down a street you notice a yellow fence. You are not measuring anything, but you are building a rough picture of where things are as you go. Later you come around a corner from a different direction and see that same fence. You have never seen it from this angle before, but you recognize it, and that tells you where you are. It also tells you that your sense of the neighborhood's layout was a little off, so you adjust it.

That recognition is the important part. Between landmarks you are doing something close to dead reckoning, keeping track of roughly how far you have walked and which way you turned, and that estimate drifts. Seeing the fence again pins you back down, and it corrects the whole map you currently have.

Cartographer does the same thing with laser scans instead of fences. It compares each new scan against what it has already recorded to work out how the robot moved. When the robot returns somewhere it has been before, it recognizes the shape of the place and adjusts the map accordingly. You may see the map shift slightly as you drive around, and that is this correction happening.

SLAM is one of the central problems in mobile robotics. Any robot that has to operate in an area without a perfect map beforehand is solving some version of this problem.

3.2 Anatomy of a Launch File

Every launch file defines one function, `generate_launch_description()`, which returns a `LaunchDescription` containing **actions**. The pieces to understand:

<code>Node(...)</code>	Start one node. Takes <code>package</code> , <code>executable</code> , and optionally <code>arguments</code> , <code>parameters</code> , <code>remappings</code> .
<code>DeclareLaunchArgument</code>	Declare a value the user can set on the command line: <code>ros2 launch pkg f.launch.py rviz:=false</code>
<code>LaunchConfiguration</code>	This reads the value of the <code>DeclareLaunchArgument</code> in question. This is what gets sent to the node
<code>IncludeLaunchDescription</code>	Run another package's launch file, optionally overriding its arguments.
<code>get_package_share_directory</code>	Find where a package's files were installed . Use this to build paths. Never hardcode <code>/opt/ros/...</code>

Note: files under `launch/`, `config/`, and `rviz/` only reach the install directory if they are listed in `data_files` in `setup.py`. This already exists in the skeleton code of the lab.

3.3 Writing `slam_launch.py`

Open `launch/slam_launch.py`. Some nodes are given as a worked example. There is also the `launch/tag_launch.py` As another example. Your launch file should start Cartographer, a node that turns its output into a map, and RViz.

Task 2: Complete the two TODOs.

1. Declare an `rviz` launch argument, defaulting to `true`.
2. Finish the RViz node. Start RViz with this package's `rviz/slam.rviz`, only when `rviz` is `true`.

When you finish the launch file, drive the robot back to the X before launching.

Build, source, and run:

```
colcon build
source install/setup.bash
ros2 launch turtlebot_controller slam_launch.py
```

RViz should open with the laser scan visible and a map filling in around the robot. Teleop the TurtleBot around the room to build a floor plan. You can also now run your `square_drive.py` to see the measurements with the support of the LiDAR!

3.4 Saving the Map

```
ros2 run nav2_map_server map_saver_cli -f ~/ros_workspaces/lab4/slam_map
```

3.5 Going Further

It should be specified that Cartographer did most of the work here, and we have treated it as a black box. It is worth knowing that its behavior is configurable rather than fixed: the file it reads controls how far the LiDAR is trusted, how consecutive scans are matched against each other, and how confident the robot must be before it decides it has returned somewhere it has been before. You can read that file at

`/opt/ros/humble/share/turtlebot3_cartographer/config/turtlebot3_lds_2d.lua` without needing the mathematics behind it.

The theory behind these **Kalman filters** (estimating a robot's position from noisy sensors, and correcting that estimate as new measurements arrive) is a subject beyond the scope of this class. Other classes like EECS106B, EECS116, EECS126, and ME233 cover this in much more depth.

Checkpoint 2

Submit a checkoff request at <https://tinyurl.com/ee106a-fa26lab>. At this point you should be able to:

- Bring up Cartographer, RViz, and the map with a single command
 - Walk through your launch file and explain what each of your additions does
 - Show your floor plan and interpret it. Why does the TurtleBot have trouble with chair and table legs? Why does it have less trouble with other TurtleBots, even moving ones?
-

4 Navigation with Nav2

So far you have driven the TurtleBot by hand, and built a map of the room. Now we will use ROS to automate this.

Nav2 is the standard navigation stack for ROS 2. It takes a map, a goal, and a stream of sensor data, and produces the velocity commands that drive the robot there. It will plan a route, adjust that route when something unexpected appears, and decide what to do when it gets stuck.

It is worth knowing that most companies building mobile robots do not run Nav2 in production; they write their own stacks, often without ROS at all. But the problems Nav2 solves are the same problems every one of those stacks has to solve. Industries will not expect you to know their stack, but they would love a close alternative.

The pieces you will see working in this section:

- **Costmaps.** When you were running RViz, you may have noticed the squares on the map. This is an occupancy grid: every cell is a range of free, occupied, or unknown. A **costmap** is the same grid with a number in each cell instead, and our planner treats a higher number as a worse place to drive. Those numbers come in layers: your saved map, obstacles the LiDAR is seeing right now, and an *inflation* layer that raises the cost of cells near obstacles without making them impassable. There is a **global** costmap covering the whole map, and a **local** one that follows the robot.
- **A global planner**, which finds a route across the global costmap.
- **A local controller**, which turns that route into velocity commands many times a second, and steers around things that were not on the map.
- **Recovery behaviors**, which run when the robot cannot make progress: clear the costmap, spin in place, back up, wait, and eventually give up.
- **AMCL**, (Adaptive Monte Carlo Localization) which figures out where the robot is on the saved map. It maintains many guesses at once and scores each by asking what the LiDAR would see from that position, narrowing them down as the robot moves. This is the same job Cartographer was doing in the last section, except against a map that already exists rather than one being built.

Most of this runs without you touching it. What you will write is the piece that tells it where to go.

4.1 Bringing up Nav2

Stop Cartographer first. Both AMCL and Cartographer publish the transform from `map` to `odom`.

Then, from the workstation:

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py \
↵ map:=$HOME/ros_workspaces/lab4/slam_map.yaml
```

Nav2 is now up, but it will not know the map frame until Section 4.6

4.2 Actions

In Lab 2 you used a **service**: you call it, you wait, you get an answer back. That works well for something quick, like spawning a turtle. Navigating across a room takes a minute or more, and a service call gives you nothing until it is over.

An **action** is the third way nodes talk to each other in ROS 2, and it is built for exactly this case. An action client sends a **goal** to an action server, and then receives:

- **feedback** while the task runs, which for navigation is how far is left to travel,
- a **result** when it finishes, including whether it succeeded or gave up,
- and the ability to **cancel** a goal that is already underway.

You could instead publish a `PoseStamped` to `/goal_pose` and the robot would drive there. But nothing comes back: no progress, no notification of arrival, no reason for failure, and no way to stop it short of `Ctrl+C`. That is why Nav2's navigation server is an action server.

The cost of this is that the call is **asynchronous**. Sending a goal does not return a result. It returns a **future**, which is a placeholder for an answer that will arrive later. You attach a callback to the future and carry on. This happens twice: once to find out whether the goal was accepted, and again to find out how it ended. The starter code handles this plumbing.

4.3 TF and Coordinate Frames

Before we can talk about where the tag is, we need to say how ROS keeps track of positions.

A **coordinate frame** is a set of axes attached to something physical: the robot's body, its camera, a point on the floor. A position is only meaningful with respect to one of them. ROS manages these relationships using **TF**, which maintains a tree of coordinate frames and the transformations between them, buffered over time.

You have already been using it. Every RViz display you have opened has a **Fixed Frame** setting, and TF is what lets RViz draw laser scans, maps, and the robot model in the same picture.

With RViz open, click **Add** in the Displays panel and add a TF display. Each set of axes is one coordinate frame. Expand

```
TF -> Frames
```

to enable or disable individual frames and make the visualization easier to read.

TF can also be queried from the command line:

```
ros2 run tf2_ros tf2_echo odom base_footprint
```

This continuously prints the transformation from `odom` to `base_footprint`, which is where the robot is relative to where it was sitting when you ran the bringup.

Who publishes what

The important thing about that command is that no single node knows the answer to it.

- The robot's description publishes the frames within the robot itself.
- `odom -> base_footprint` comes from the wheel encoders, the same odometry you were reading earlier.
- Nav2 will publish `map -> odom` once it has been told roughly where the robot is on your saved map. Nothing has told it yet, so the `map` frame does not exist at the moment.
- A static transform, which you will run in a moment, publishes where the camera sits on the robot.
- The tag detector publishes each tag it sees, relative to the camera.

TF assembles these into a single tree. As long as two frames are connected within that tree, you can ask for the transform between them, and TF composes the intermediate steps for you.

The query looks like this:

```
tf_buffer.lookup_transform(  
    target_frame,  
    source_frame,  
    rclpy.time.Time()  
)
```

The returned value is a `geometry_msgs/TransformStamped` message containing the translation and rotation between the two frames. Its translation is the origin of `source_frame` expressed in `target_frame`.

A `tf2_ros.Buffer` stores the transforms currently known to TF, and a `tf2_ros.TransformListener` keeps it filled. Both are created for you in the starter code. If you have already done Lab 3, this is the same machinery you used there.

4.4 Finding a Goal with ArUco Tags

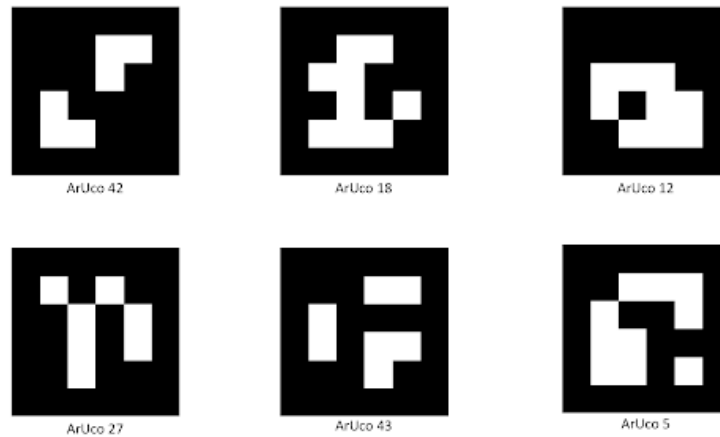


Figure 1: Example ArUco Tags

ArUco tags are square patterns printed on a flat surface, like the ones in Figure 1. Their corners are easy to identify from a single camera perspective, so the position of a tag can be inferred from its expected size and the skew detected in the image. The pattern at the centre of the tag identifies which tag it is, so several can be told apart in one image.

When the camera is calibrated and the size of the marker is known, the pose of the tag can be computed in real distance units. Detection depends on three things:

1. **The image** (`/image_raw`), from the camera on the TurtleBot.
2. **The camera's intrinsic parameters** (`/camera_info`), which describe how 3D points project onto the 2D image. The pose cannot be recovered correctly without them.
3. **The tag size**, 15 cm $\times$ 15 cm in this lab. If this is wrong, every distance the detector reports is wrong by the same factor.

4.5 TF Frame Broadcasting

As well as publishing poses, the detector broadcasts a **TF frame** for each tag it sees. Every tag becomes a coordinate frame in the TF tree, named by its ID:

`ar_marker_7`

for the tag with ID 7. These frames are children of the camera frame, and can be visualised in RViz like any other frame. **Your ArUco tag will be the ID it is labeled with**

With that link in place the chain is complete:

```
map -> odom -> base_footprint -> base_link -> camera -> ar_marker_7
```

AMCL publishes the first link, the robot publishes the middle ones, the static transform publishes the camera link, and the detector publishes the last. No single node knows the whole chain, and none of them needs to. One call walks all of it:

```
trans = tfBuffer.lookup_transform('map', 'ar_marker_7', rclpy.time.Time())
```

`lookup_transform(target, source, time)` gives the transform from `target` to `source`; its translation is the origin of `source` expressed in `target`. So this returns the tag's position in map coordinates, which is the frame a navigation goal is specified in.

4.6 Running the Tag Detector

Your tag has an ID printed on it. Substitute yours for 7 below.

The camera is already running from the initial bringup (if not rerun the camera bringup). On the workstation:

```
ros2 launch turtlebot_controller tag_launch.py
```

This starts two things: the fixed `base_link` → `camera` transform, and the detector itself.

Place the tag in front of the TurtleBot and check:

```
ros2 topic echo /aruco_markers
ros2 run tf2_ros tf2_echo camera ar_marker_7
```

1. `/aruco_markers` publishes only while a tag is actually in frame. Nothing arriving means the detector cannot see it. Check lighting, angle, and distance.
2. `tf2_echo` reports the tag's position in the camera frame, where `z` is the forward distance. For reasons that will be covered in a later content lecture it will not be perfect, but the reported distance should be fairly close.

4.7 Writing `nav_client.py`

Open `nav_client.py` in the `turtlebot_controller` package. The node already seeds AMCL, waits for the TF buffer to fill, and handles the result of the goal. You will fill in six TODOs.

Task 3: Complete `nav_client.py`.

1. **Build the action client.** You need the action type and the name of the server. Both are visible from the command line while Nav2 is running:

```
ros2 action list -t
```

2. **Look up the tag.** One `lookup_transform` call, from `self.goal_frame` to the marker's frame. TF composes the whole chain for you. The staleness check below it is already written. Read it, since it is the difference between tracking a tag and chasing a ghost.
3. **Work out where to stand.** The tag is on a wall or a cone, so its own position is inside an obstacle and Nav2 will refuse to plan there. Return a goal `self.standoff` metres short of the tag, along the line from the robot to the tag, facing it.
4. **Build the PoseStamped.** It needs a `frame_id`, a stamp, a position, and an orientation. `yaw_to_quaternion` is given at the top of the file. An all-zero quaternion is not valid, and a goal carrying one will be rejected.
5. **Send the goal.** Sending returns a future rather than a result. Attach `on_goal_response` to it, and pass `on_feedback` as the feedback callback. `on_goal_response` is already written and does the same thing a second time to get the result. Read it first.
6. **Print the progress.** Nav2's feedback carries how far it thinks it still has to travel. You may want to throttle your printing.

Before moving to the marker, run this first. As well as skipping the tag, this run is what tells Nav2 where the robot is. The `map` frame does not exist until something seeds it, so the tag lookup has nothing to resolve against until this has been run once.

```
ros2 run turtlebot_controller nav_client --ros-args -p x:=1.5 -p y:=0.0
```

Once that works, drop the coordinates and let it find the tag:

```
ros2 run turtlebot_controller nav_client --ros-args -p marker_id:=7
```

4.8 Watching it Drive

Nav2 publishes most of its internal state, so you can watch it think. In RViz, add displays for:

- `/global_costmap/costmap`, which is the map you saved plus a colored band around every wall. That band is the inflation layer: cells near obstacles are expensive rather than forbidden. It is why the robot will drive down the middle of a corridor instead of cutting the corner.
- `/plan`, the route across the global costmap.
- `/local_costmap/costmap`, a small window that follows the robot, built from what the LiDAR sees right now.
- `/local_plan`, the short curve the robot is actually following, redrawn several times a second.

Send the goal again and watch which of these changes, and when.

4.9 Something in the Way

Now put an obstacle between the robot and its goal. A traffic cone works well, as we just need something tall enough for the LiDAR to detect it.

Send the goal with the cone in place. It was not there when you built the map, so the global costmap knows nothing about it, but the LiDAR sees it and the robot steers around it anyway. Watch which costmap lights up.

You can also place the cone mid-drive! Watch its behavior.

Checkpoint 3

Remember to `colcon build` and `source install/setup.bash` after editing `nav_client.py`.

Submit a checkoff request at <https://tinyurl.com/ee106a-fa261ab>. At this point you should be able to:

- Command the TurtleBot to the tag from your own code, without clicking in RViz
 - Explain why navigation is an action rather than a service, and name one thing an action gives you that publishing a goal pose would not
 - List the frames your `lookup_transform` call traversed, and say what publishes each link
 - Which costmap knew about the cone? Why did the shorter object get run over?
-

5 Before you Leave

Save your code:

```
cd ~/ros_workspaces
git add -A lab4
git commit -m "Complete Lab 4"
git push origin main
```

The lab workstations and the TurtleBots are shared. Before you go:

- **Ctrl+C** out of every terminal. Do **not** use **Ctrl+Z**, which only pauses a process and leaves it running in the background.
- Nav2 does not always shut down immediately on **Ctrl+C**. Give its terminal a few seconds to finish, and check that it has:

```
ros2 node list
```

If Nav2 nodes are still listed after everything is closed, tell a TA rather than trying to clear them yourself.

- **Ctrl+C** the bringup in your ssh session and type **exit**. If you leave it running, the robot keeps publishing to the network after you have gone.
- Turn the TurtleBot off and plug its battery back into the charging station.
- **exit** your distrobox container, and log out of your **ee106a-*** account.

Congratulations on finishing Lab 4!